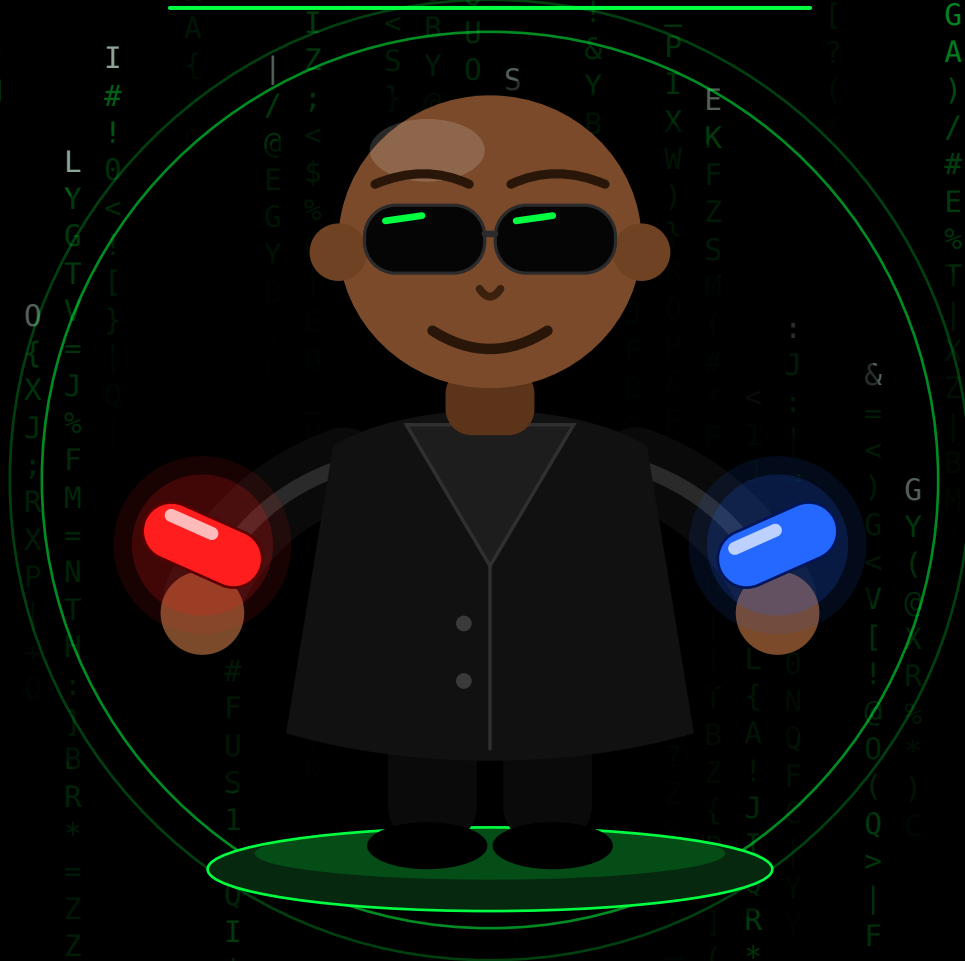


ESTUDO · GR // APOSTILA DE BOLSO

DADOS COM PYTHON



Teoria, receitas e exercícios para relembrar
como obter resultados com código — sem decorar, entendendo.

01 Python básico

02 POO

03 Pandas

04 SQLite & SQL

05 Tkinter

06 Projeto integrador

Como usar esta apostila

Esta apostila foi feita para **relembrar como obter resultados** com Python: cada capítulo explica a teoria, mostra receitas com exemplos *diferentes* dos do ambiente de prática, traz um exemplo resolvido passo a passo e termina com exercícios para você resolver diante do computador, na aba **Apostila** do app.

Como cada capítulo está organizado

- **Teoria e receitas** — conceito curto + código que você pode copiar e adaptar. Cada bloco com saída mostra o resultado **real** da execução.
- **Exemplo resolvido** — do enunciado ao código, pensando em voz alta.
- **Armadilhas** — os tropeços mais comuns.
- **Cola rápida** — tabela de consulta de bolso.
- **Exercícios** — enunciado, ponto de partida e um espaço de rascunho. As soluções comentadas estão no **gabarito** no fim (tente antes de olhar!).

Usando a aba Apostila do app

1. Abra **estudogr.vercel.app** e escolha a **pílula vermelha** (login com Google) para que seu progresso e seu código fiquem salvos no banco de dados.
2. No topo da barra lateral há duas abas: **Exercícios** (os 16 do ambiente) e **Apostila** (os 16 exercícios deste material, na mesma ordem dos capítulos).
3. Clique em um exercício, leia o enunciado, escreva no editor e use **Executar** para ver a saída e **Verificar** para a correção automática.
4. Ao acertar, aparece a solução comentada ('versão otimizada') para você comparar com a sua.
5. O código que você escreve é salvo sozinho (e no banco, se estiver logado). Pode fechar e voltar depois.
6. O botão **Baixar apostila (PDF)** no cabeçalho baixa este material.

Sumário

Como usar esta apostila	2
1. Python básico	5
1.1 Variáveis e tipos de dados	5
1.2 Listas, tuplas, dicionários e conjuntos	5
1.3 Condicionais e laços	6
1.4 Comprehensions e funções embutidas	7
1.5 Funções	7
1.6 Textos (strings) e f-strings	8
1.7 Erros e exceções	8
Exemplo resolvido — fechamento do caixa da padaria	9
Exercícios do capítulo 1	10
2. Programação orientada a objetos	12
2.1 Classe, objeto, <code>__init__</code> e <code>self</code>	12
2.2 Atributos de classe e métodos especiais	12
2.3 Herança e polimorfismo	13
2.4 Encapsulamento: proteger o estado	14
Exemplo resolvido — fila do banco de sangue	14
Exercícios do capítulo 2	15
3. Pandas — análise de dados	18
3.1 Criar e inspecionar um DataFrame	18
3.2 Selecionar e filtrar	18
3.3 Colunas novas, ordenar e renomear	19
3.4 Agrupar e resumir (groupby)	19
3.5 Limpeza de dados	20
3.6 Tabela dinâmica (pivot_table)	21
3.7 Juntar tabelas (merge e concat)	21
Exemplo resolvido — a artilharia do campeonato	22
Exercícios do capítulo 3	23
4. SQLite e SQL — banco de dados	26
4.1 Conceitos e o módulo <code>sqlite3</code>	26
4.2 Consultar: <code>SELECT</code> , <code>WHERE</code> , <code>ORDER BY</code> , <code>LIMIT</code>	27
4.3 Alterar dados: <code>UPDATE</code> e <code>DELETE</code>	27
4.4 Agregações: <code>COUNT</code> , <code>SUM</code> , <code>AVG</code> , <code>GROUP BY</code> e <code>HAVING</code>	27

4.5 Juntar tabelas: JOIN	28
4.6 Pandas + SQL	29
Exemplo resolvido — ranking de peças da oficina	29
Exercícios do capítulo 4	30
5. Tkinter — interfaces gráficas	33
5.1 Estrutura mínima de uma janela	33
5.2 Widgets mais usados	33
5.3 Posicionamento: pack, grid e place	33
5.4 Eventos e callbacks	34
5.5 Organizando o app em uma classe	34
Exemplo resolvido — checklist de viagem	34
Exercícios do capítulo 5	35
6. Projeto integrador	37
6.1 Arquitetura em camadas	37
6.2 Roteiro para qualquer projeto novo	37
6.3 Esqueleto completo (estúdio de música)	37
Exercícios do capítulo 6	39
Apêndice A — Método para resolver qualquer exercício	40
Apêndice B — Mensagens de erro e o que fazer	40
Gabarito comentado	41
Capítulo 1 — Python básico	41
Capítulo 2 — Programação orientada a objetos	42
Capítulo 3 — Pandas — análise de dados	43
Capítulo 4 — SQLite e SQL — banco de dados	44
Capítulo 5 — Tkinter — interfaces gráficas	45
Capítulo 6 — Projeto integrador	46

CAPÍTULO 01

Python básico

Tudo nos capítulos seguintes (classes, pandas, SQL) é construído sobre este alicerce: guardar valores, escolher caminhos, repetir tarefas e empacotar lógica em funções. Se você dominar estas páginas, o resto vira combinação de peças.

1.1 Variáveis e tipos de dados

Uma **variável** é um nome que aponta para um valor. Em Python você não declara o tipo: ele vem do valor. Os tipos básicos são **int** (inteiro), **float** (decimal), **str** (texto), **bool** (verdadeiro/falso) e **None** (ausência de valor).

Tipos e conversões

```
pao_unidade = 0.75      # float
quantidade = 12         # int
padaria = "Pão Quente"  # str
aberta = True           # bool
cliente = None          # None: ainda não temos cliente

print(type(pao_unidade), type(quantidade), type(padaria))
total = pao_unidade * quantidade
print(f"{padaria}: {quantidade} pães = R$ {total:.2f}")
print(int("42") + 1, float("3.5") * 2, str(7) + "!")
```

► SAÍDA

```
<class 'float'> <class 'int'> <class 'str'>
Pão Quente: 12 pães = R$ 9.00
43 7.0 7!
```

LEMBRE Conversões: **int()**, **float()**, **str()**. O resultado de **input()** é sempre texto: converta antes de fazer conta.

CUIDADO = atribui um valor; **==** compara dois valores. Confundir os dois é um dos erros mais comuns de quem está começando.

1.2 Listas, tuplas, dicionários e conjuntos

São as quatro formas de guardar *coleções*. Escolha pela pergunta que você vai fazer aos dados:

Tipo	Sintaxe	Use quando...	Mutável?
list	[1, 2, 3]	ordem importa e pode repetir	sim
tuple	(1, 2)	grupo fixo (ex.: par nome/valor)	não
dict	{"a": 1}	buscar valor por uma chave	sim
set	{1, 2, 3}	sem repetidos / testar pertencimento	sim

Operações essenciais

```
padaria = {"nome": "Pão Quente", "horario": (6, 20)}
paes = ["francês", "integral", "brioche"]

paes.append("baguete")          # adiciona no fim
print(paes[0], paes[-1])       # primeiro e último
print(paes[1:3])               # fatia: posições 1 e 2
print(padaria["nome"], padaria.get("telefone", "sem telefone"))

padaria["telefone"] = "9999-0000" # adiciona/atualiza chave
for chave, valor in padaria.items(): # percorre pares
    print(chave, "->", valor)

print(set([1, 2, 2, 3, 3, 3]))   # remove repetidos
```

► SAÍDA

```
francês baguete
['integral', 'brioche']
Pão Quente sem telefone
nome -> Pão Quente
horario -> (6, 20)
telefone -> 9999-0000
{1, 2, 3}
```

DICA dict.get(chave, padrao) não dá erro se a chave não existir; dict[chave] dá KeyError. Use get quando a chave pode faltar.

1.3 Condicionais e laços

if / elif / else escolhem um caminho. **for** repete para cada item de uma sequência; **while** repete enquanto uma condição for verdadeira. A **indentação** (4 espaços) define o que está dentro do bloco.

if/elif/else, for, enumerate, zip

```
vendas_dia = [120.0, 85.5, 0, 240.0, 60.0]

for i, valor in enumerate(vendas_dia, start=1): # i começa em 1
    if valor == 0:
        print(f"Dia {i}: loja fechada")
        continue                               # pula para o próximo
    elif valor >= 100:
        faixa = "bom"
    else:
        faixa = "fraco"
    print(f"Dia {i}: R$ {valor:.2f} ({faixa})")

nomes = ["Ana", "Bia"]
notas = [9, 7]
for nome, nota in zip(nomes, notas):           # percorre em paralelo
    print(nome, nota)
```

► SAÍDA

```
Dia 1: R$ 120.00 (bom)
Dia 2: R$ 85.50 (fraco)
Dia 3: loja fechada
Dia 4: R$ 240.00 (bom)
Dia 5: R$ 60.00 (fraco)
Ana 9
Bia 7
```

- **range(n)**: 0 até n-1 (range(1, 6) vai de 1 a 5).
- **break** sai do laço; **continue** pula para a próxima volta.
- **and / or / not** combinam condições; **in** testa pertencimento (*if 'Ana' in nomes*).

1.4 Comprehensions e funções embutidas

Uma **list comprehension** cria uma lista nova a partir de outra em uma linha: *[expressão for item in sequência if condição]*. As funções embutidas **sum**, **min**, **max**, **len**, **sorted**, **any**, **all** resolvem a maioria dos laços simples.

sum, max(key=), sorted(key=), any/all

```
pecas = [
    {"nome": "Filtro de óleo", "preco": 35.0, "qtd": 10},
    {"nome": "Pastilha de freio", "preco": 120.0, "qtd": 4},
    {"nome": "Lâmpada", "preco": 12.5, "qtd": 30},
]

valor_total = sum(p["preco"] * p["qtd"] for p in pecas)
mais_cara = max(pecas, key=lambda p: p["preco"])
baratas = [p["nome"] for p in pecas if p["preco"] < 40]
por_qtd = sorted(pecas, key=lambda p: p["qtd"], reverse=True)

print(valor_total, mais_cara["nome"], baratas)
print([p["nome"] for p in por_qtd])
print(any(p["qtd"] < 5 for p in pecas), all(p["preco"] > 10 for p in pecas))
quadrados = {n: n ** 2 for n in range(1, 5)}      # dict comprehension
print(quadrados)
```

► SAÍDA

```
1205.0 Pastilha de freio ['Filtro de óleo', 'Lâmpada']
['Lâmpada', 'Filtro de óleo', 'Pastilha de freio']
True True
{1: 1, 2: 4, 3: 9, 4: 16}
```

LEMBRE `key=` diz *por qual critério* comparar. `lambda p: p["preco"]` é uma função de uma linha que devolve o preço.

1.5 Funções

Uma função empacota uma lógica com nome, recebe **parâmetros** e devolve um resultado com **return**. Sem `return`, devolve `None`. Parâmetros podem ter **valor padrão**, e dá para devolver vários valores (na verdade, uma tupla).

def, return, parâmetro padrão, múltiplos retornos

```
def calcula_frete(peso_kg, expresso=False):
    """Frete: R$ 8 por kg; expresso custa 50% a mais."""
    valor = peso_kg * 8
    if expresso:
        valor *= 1.5
    return round(valor, 2)

def resumo(valores):
    return min(valores), max(valores), sum(valores) / len(valores)

print(calcula_frete(2))                # usa o padrão (expresso=False)
print(calcula_frete(2, expresso=True)) # argumento nomeado
menor, maior, media = resumo([10, 20, 60]) # desempacotamento
print(menor, maior, round(media, 1))
```

► SAÍDA

```
16
24.0
10 60 30.0
```

CUIDADO Variáveis criadas *dentro* da função só existem lá (escopo local). Para usar um valor de fora, passe-o como parâmetro; evite *global*.

1.6 Textos (strings) e f-strings

Strings têm dezenas de métodos prontos. Os que mais aparecem em tratamento de dados: **strip**, **lower**, **upper**, **title**, **replace**, **split**, **join**, **startswith**, **endswith**, **isdigit**, **in**. As **f-strings** inserem valores no texto, com formatação depois dos dois pontos.

Métodos de texto e formatação

```
codigo = " pet-2024-0087 "  
limpo = codigo.strip().upper()  
print(limpo, limpo.startswith("PET"), limpo.split("-"))  
print("-".join(["a", "b", "c"]), "gato preto".title(), "2024".isdigit())  
  
preco, qtd = 1234.5, 3  
print(f"Total: R$ {preco * qtd:,.2f}")      # milhar e 2 casas  
print(f"{'Item':<10}|{'Qtd':>5}")          # alinhar à esq./dir.  
print(f"{0.256:.1%} {7:03d}")              # percentual e zeros à esquerda
```

► SAÍDA

```
PET-2024-0087 True ['PET', '2024', '0087']  
a-b-c Gato Preto True  
Total: R$ 3,703.50  
Item      | Qtd  
25.6%    007
```

Formato	Efeito	Exemplo → resultado
:.2f	2 casas decimais	f'{3.14159:.2f}' → 3.14
:,	separador de milhar	f'{1234567:,}' → 1,234,567
:>8	alinha à direita em 8 espaços	f'{42:>8}' → ' 42'
:.1%	percentual	f'{0.256:.1%}' → 25.6%

1.7 Erros e exceções

Quando algo dá errado, Python *levanta uma exceção*. Você pode antecipar com **try/except** e também **levantar** a sua própria com **raise** para recusar dados inválidos.

try / except / finally e raise

```
def divide(a, b):  
    if b == 0:  
        raise ValueError("divisor não pode ser zero")  
    return a / b  
  
for dado in [(10, 2), (1, 0)]:  
    try:  
        print("resultado:", divide(*dado))  
    except ValueError as erro:  
        print("recusado:", erro)  
    finally:  
        print("-- fim da tentativa --")
```

► SAÍDA

```
resultado: 5.0  
-- fim da tentativa --  
recusado: divisor não pode ser zero  
-- fim da tentativa --
```

Erro	Quase sempre significa...
SyntaxError / IndentationError	falta '.', parêntese aberto ou indentação errada
NameError	usou um nome que ainda não existe (ou escreveu errado)
TypeError	misturou tipos (ex.: texto + número)

Erro	Quase sempre significa...
KeyError	a chave do dicionário/coluna não existe
IndexError	posição fora da lista
ValueError	valor com tipo certo, mas conteúdo inválido (int('abc'))
ZeroDivisionError	divisão por zero

Exemplo resolvido — fechamento do caixa da padaria

ENUNCIADO A lista abaixo tem as vendas do dia (item, preço, quantidade). Descubra o faturamento total, o item mais vendido (em quantidade) e a lista de itens que venderam menos de 5 unidades, em ordem alfabética.

Como pensar:

1. **Entender os dados:** lista de dicionários; cada venda tem 3 campos.
2. **Faturamento:** soma de preço × quantidade → sum() com expressão geradora.
3. **Mais vendido:** max() com key=quantidade, depois pegar o campo 'item'.
4. **Poucos vendidos:** comprehension com if, e sorted() para ordenar.

Solução completa

```
vendas = [
    {"item": "pão francês", "preco": 0.75, "qtd": 200},
    {"item": "bolo de milho", "preco": 18.0, "qtd": 3},
    {"item": "café", "preco": 4.5, "qtd": 60},
    {"item": "croissant", "preco": 7.0, "qtd": 4},
]

faturamento = sum(v["preco"] * v["qtd"] for v in vendas)
mais_vendido = max(vendas, key=lambda v: v["qtd"])["item"]
poucos = sorted(v["item"] for v in vendas if v["qtd"] < 5)

print(f"Faturamento: R$ {faturamento:.2f}")
print("Mais vendido:", mais_vendido)
print("Poucos vendidos:", poucos)
```

► SAÍDA

```
Faturamento: R$ 502.00
Mais vendido: pão francês
Poucos vendidos: ['bolo de milho', 'croissant']
```

Armadilhas comuns

- ! Esquecer o : no fim de if/for/def/while.
- ! Alterar uma lista enquanto percorre a mesma lista (crie uma nova no lugar).
- ! Comparar texto com número ('5' == 5 é False). Converta com int()/float().
- ! Usar **round()** no meio dos cálculos: arredonde só no resultado final.

Cola rápida

Quero...	Use...
somar / contar / maior	<code>sum(...)</code> / <code>len(...)</code> / <code>max(..., key=...)</code>
filtrar e transformar	<code>[expr for x in lista if cond]</code>
ordenar por um campo	<code>sorted(lista, key=lambda x: x['campo'])</code>
percorrer com posição	<code>for i, x in enumerate(lista):</code>
valor com padrão	<code>dicionario.get(chave, padrao)</code>
limpar texto	<code>texto.strip().lower()</code>
recusar dado inválido	<code>raise ValueError('mensagem')</code>

Exercícios do capítulo 1

Resolva no app (aba **Apostila**) e confira com a correção automática. Antes de digitar, use o rascunho para anotar entrada, saída e passos.

Exercício 1 — Acervo da biblioteca de bairro

[Apostila > ap_acervo](#)

Você cuida do acervo de uma biblioteca. A lista "livros" tem, para cada livro, 'titulo', 'paginas' e 'exemplares'.

Calcule:

- `total_paginas`: soma de paginas * exemplares de todos os livros;
- `livro_mais_longo`: título do livro com mais páginas;
- `titulos_raros`: lista com os títulos que têm apenas 1 exemplar (na ordem original).

Ponto de partida (já vem pronto no editor do app):

```
livros = [
    {"titulo": "Dom Casmurro", "paginas": 256, "exemplares": 3},
    {"titulo": "O Cortiço", "paginas": 320, "exemplares": 1},
    {"titulo": "Memórias Póstumas", "paginas": 288, "exemplares": 2},
    {"titulo": "Iracema", "paginas": 176, "exemplares": 1},
]

# TODO: soma de paginas * exemplares
total_paginas = 0

# TODO: título do livro com mais páginas
livro_mais_longo = ""

# TODO: lista de títulos com exatamente 1 exemplar
titulos_raros = []
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 2 — Validador de matrícula escolar

[Apostila > ap_matricula](#)

Uma escola usa matrículas no formato 'MAT-2024-0153': prefixo MAT, ano com 4 dígitos e número com 4 dígitos, separados por hífen.

Escreva a função `matricula_valida(texto)` que devolve `True` ou `False`.

Regras: ignore espaços nas pontas e aceite minúsculas ('mat-2024-0153' é válida).

Ponto de partida (já vem pronto no editor do app):

```
def matricula_valida(texto):  
    # TODO: devolva True se o texto seguir o formato MAT-AAAA-NNNN  
    return False
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 3 — Boletim escolar com funções e f-strings

Apostila ► [ap_boletim](#)

Escreva a função `situacao(notas)` que recebe uma lista de notas e devolve a tupla (media, texto):

- media arredondada com 1 casa decimal;
- texto = "Aprovado" se media ≥ 7 , "Recuperação" se media ≥ 5 , senão "Reprovado".

Depois, usando o dicionário "alunos", monte o dicionário "boletim" no formato {nome: texto_da_situacao}.

Ponto de partida (já vem pronto no editor do app):

```
def situacao(notas):  
    # TODO: calcule a média (1 casa decimal) e decida o texto  
    return (0, "")  
  
alunos = {  
    "Ana": [8, 9, 7.5],  
    "Bruno": [5, 6, 4],  
    "Carla": [3, 4, 2],  
}  
  
# TODO: {nome: texto} usando a função situacao  
boletim = {}
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

CAPÍTULO 02

Programação orientada a objetos

POO organiza o programa em **objetos** que juntam dados (atributos) e ações (métodos). Em vez de funções soltas manipulando dicionários, você cria o conceito (Animal, Pedido, Conta) uma vez e gera quantos exemplares precisar.

2.1 Classe, objeto, `__init__` e `self`

A **classe** é o molde; o **objeto** é cada peça fabricada com ele. O método `__init__` roda quando o objeto nasce e guarda os dados iniciais. **self** é o próprio objeto: todo método recebe `self` como primeiro parâmetro.

Classe Paciente

```
class Paciente:
    """Um animal atendido na clínica veterinária."""

    def __init__(self, nome, especie, peso_kg):
        self.nome = nome          # atributos de instância
        self.especie = especie
        self.peso_kg = peso_kg
        self.vacinas = []

    def vacinar(self, vacina):
        self.vacinas.append(vacina)

    def dose_diaria(self):
        return round(self.peso_kg * 0.5, 1)    # 0,5 mg por kg

    def __str__(self):
        # usado por print()
        return f"{self.nome} ({self.especie}, {self.peso_kg} kg)"

rex = Paciente("Rex", "cão", 12.0)
mia = Paciente("Mia", "gato", 4.2)
rex.vacinar("raiva")
print(rex, "| vacinas:", rex.vacinas)
print(mia, "| dose:", mia.dose_diaria(), "mg")
print(mia.vacinas)          # cada objeto tem a sua própria lista
```

► SAÍDA

```
Rex (cão, 12.0 kg) | vacinas: ['raiva']
Mia (gato, 4.2 kg) | dose: 2.1 mg
[]
```

CUIDADO Esqueceu o `self` no `def`, ou o `self` antes do atributo? Vai aparecer `TypeError` ou `NameError`. Dentro da classe, atributos sempre levam `self`.

2.2 Atributos de classe e métodos especiais

Um **atributo de classe** é compartilhado por todos os objetos (ex.: uma taxa fixa). `@classmethod` cria objetos por caminhos alternativos; `@staticmethod` é uma função utilitária dentro da classe. Métodos *dunder* (`__str__`, `__repr__`, `__eq__`, `__lt__`, `__len__`) ensinam o Python a imprimir, comparar e medir seus objetos.

@classmethod, atributo de classe, __lt__ e __repr__

```
class Consulta:
    taxa_clinica = 15.0                # atributo de CLASSE

    def __init__(self, paciente, valor):
        self.paciente = paciente
        self.valor = valor

    @classmethod
    def de_texto(cls, texto):          # "Rex;120"
        paciente, valor = texto.split(";")
        return cls(paciente, float(valor))

    def total(self):
        return self.valor + self.taxa_clinica

    def __lt__(self, outro):           # permite sorted()/min()/max()
        return self.total() < outro.total()

    def __repr__(self):
        return f"Consulta({self.paciente!r}, {self.total()})"

consultas = [Consulta.de_texto("Rex;120"), Consulta("Mia", 80)]
print(sorted(consultas))
print(max(consultas))
```

► SAÍDA

```
[Consulta('Mia', 95.0), Consulta('Rex', 135.0)]
Consulta('Rex', 135.0)
```

2.3 Herança e polimorfismo

Herança: uma classe-filha reaproveita tudo da classe-mãe e acrescenta ou *sobrescreve* o que for diferente. **super()** chama a versão da mãe. **Polimorfismo:** você chama o mesmo método em objetos de tipos diferentes e cada um responde do seu jeito.

Atendimento → Emergencia e Retorno

```
class Atendimento:
    def __init__(self, paciente, base):
        self.paciente = paciente
        self.base = base

    def valor(self):
        return self.base

class Emergencia(Atendimento):
    def valor(self):
        return super().valor() * 2        # dobro do valor base

class Retorno(Atendimento):
    def valor(self):
        return super().valor() * 0.5      # metade

fila = [Atendimento("Rex", 100), Emergencia("Mia", 100), Retorno("Bob", 100)]
for a in fila:
    # polimorfismo em ação
    print(f"{a.paciente:<4} {type(a).__name__:<12} R$ {a.valor():.2f}")
print("Total:", sum(a.valor() for a in fila))
print(isinstance(fila[1], Atendimento), isinstance(Retorno, Emergencia))
```

► SAÍDA

```
Rex  Atendimento  R$ 100.00
Mia  Emergencia   R$ 200.00
Bob  Retorno      R$ 50.00
Total: 350.0
True False
```

LEMBRE Quando usar herança? Quando existe a relação 'é um tipo de' (Emergência é um tipo de Atendimento). Se for 'tem um' (Clínica tem pacientes), use composição: um atributo guardando outros objetos.

2.4 Encapsulamento: proteger o estado

Encapsular é controlar *como* os dados mudam. Por convenção, nomes com underscore (`_saldo`) são de uso interno. Uma `@property` expõe o valor como se fosse atributo (somente leitura). Com um **setter** você valida antes de aceitar a mudança.

property com setter e validação

```
class Termometro:
    def __init__(self, temp):
        self._temp = None
        self.temperatura = temp          # passa pelo setter (valida)

    @property
    def temperatura(self):
        return self._temp

    @temperatura.setter
    def temperatura(self, valor):
        if not -50 <= valor <= 150:
            raise ValueError("temperatura fora da faixa")
        self._temp = valor

    @property
    def fahrenheit(self):                # propriedade calculada
        return self._temp * 9 / 5 + 32

t = Termometro(36.5)
print(t.temperatura, round(t.fahrenheit, 1))
t.temperatura = 40
try:
    t.temperatura = 900
except ValueError as erro:
    print("recusado:", erro, "| continua:", t.temperatura)
```

► SAÍDA

```
36.5 97.7
recusado: temperatura fora da faixa | continua: 40
```

DICA Valide **antes** de alterar o estado. Assim, uma operação recusada nunca deixa o objeto 'pela metade'.

Exemplo resolvido — fila do banco de sangue

ENUNCIADO Crie a classe `Doador(nome, tipo, litros_doados)` com o método `pode_doar_para(tipo_receptor)` usando a regra simplificada: 'O' doa para todos, 'A' doa para A e AB, 'B' para B e AB, 'AB' só para AB. Depois, dada uma lista de doadores, mostre quantos litros são compatíveis com um receptor tipo 'A'.

Como pensar:

1. **Modelar:** o que um `Doador` sabe (nome, tipo, litros) e faz (checar compatibilidade).
2. **Regra em dados:** um dicionário `tipo → conjunto de receptores` evita `if` em cascata.
3. **Usar:** filtrar com `comprehension` chamando o método do objeto.

Solução completa

```
class Doador:
    COMPATIVEL = {"O": {"O", "A", "B", "AB"}, "A": {"A", "AB"},
                  "B": {"B", "AB"}, "AB": {"AB"}}

    def __init__(self, nome, tipo, litros_doados):
        self.nome = nome
        self.tipo = tipo
        self.litros_doados = litros_doados

    def pode_doar_para(self, tipo_receptor):
        return tipo_receptor in self.COMPATIVEL[self.tipo]

doadores = [Doador("Lia", "O", 0.45), Doador("Davi", "B", 0.40),
            Doador("Iara", "A", 0.50), Doador("Teo", "AB", 0.45)]

aptos = [d for d in doadores if d.pode_doar_para("A")]
print([d.nome for d in aptos], round(sum(d.litros_doados for d in aptos), 2))
```

► SAÍDA

```
['Lia', 'Iara'] 0.95
```

Armadilhas comuns

- ! Atributo mutável como valor padrão: `def __init__(self, itens=[])` compartilha a MESMA lista entre objetos. Use `itens=None` e crie a lista dentro.
- ! Esquecer `super().__init__(...)` numa subclasse que define o seu próprio `__init__`.
- ! Chamar um método sem parênteses (`obj.total` em vez de `obj.total()`).
- ! Misturar impressão e cálculo no mesmo método: devolva o valor com `return` e imprima fora.

Cola rápida

Quero...	Use...
guardar dados iniciais	<code>def __init__(self, ...): self.x = x</code>
texto bonito no <code>print()</code>	<code>def __str__(self): return f'...'</code>
reaproveitar a classe-mãe	<code>class Filha(Mae): / super().metodo()</code>
atributo somente leitura	<code>@property + atributo _interno</code>
recusar valor inválido	<code>raise ValueError('mensagem')</code>
ordenar objetos	<code>sorted(objs, key=lambda o: o.campo)</code>
chechar o tipo	<code>isinstance(obj, Classe)</code>

Exercícios do capítulo 2

Resolva no app (aba **Apostila**) e confira com a correção automática. Antes de digitar, use o rascunho para anotar entrada, saída e passos.

Exercício 4 — Classe Livro com empréstimo e devolução

Apostila ► [ap_livro_classe](#)

Crie a classe `Livro` com:

- `__init__(self, titulo, exemplares)` guardando os dois valores;
- `emprestar()`: diminui 1 exemplar; se não houver nenhum disponível, levanta `ValueError("sem exemplares")`;
- `devolver()`: aumenta 1 exemplar;
- `__str__`: devolve "Título (N disponíveis)", por exemplo "Iracema (1 disponíveis)".

Ponto de partida (já vem pronto no editor do app):

```
class Livro:
    # TODO: __init__, emprestar, devolver e __str__
    pass
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 5 — Planos de academia com herança

Apostila ► [ap_planos_heranca](#)

A classe Plano já existe: guarda 'aluno' e 'base' e sua mensalidade() devolve a base.

Crie:

- PlanoAnual(Plano): mensalidade() com 20% de desconto sobre a base;
- PlanoPersonal(Plano): mensalidade() = base + 150 (taxa do personal).

Depois calcule total_mensal: soma de mensalidade() de todos os planos da lista "planos".

Ponto de partida (já vem pronto no editor do app):

```
class Plano:
    def __init__(self, aluno, base):
        self.aluno = aluno
        self.base = base

    def mensalidade(self):
        return self.base

# TODO: PlanoAnual (20% de desconto) e PlanoPersonal (base + 150)
class PlanoAnual(Plano):
    pass

class PlanoPersonal(Plano):
    pass

planos = [Plano("Rita", 120), PlanoAnual("Caio", 120), PlanoPersonal("Lia", 120)]

# TODO: some a mensalidade de todos os planos
total_mensal = 0
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 6 — Carteirinha de cafeteria (encapsulamento)

Apostila ► [ap_carteirinha](#)

Crie a classe `Carteirinha(dono, saldo=0)` que guarda o saldo de forma protegida:

- o saldo fica em `self._saldo` e é lido pela property `"saldo"` (somente leitura: atribuir `carteirinha.saldo = 10` deve dar erro);
- `depositar(valor)`: valor deve ser > 0 , senão `ValueError`;
- `pagar(valor)`: se `valor > saldo`, levanta `ValueError("saldo insuficiente")`; senão desconta.

Ponto de partida (já vem pronto no editor do app):

```
class Carteirainha:
    # TODO: __init__, property saldo, depositar e pagar
    pass
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

CAPÍTULO 03

Pandas — análise de dados

Pandas trabalha com tabelas: o **DataFrame** (linhas × colunas) e a **Series** (uma coluna). A ideia central é *operar na coluna inteira de uma vez*, sem escrever laços. Convenção universal: *import pandas as pd*.

3.1 Criar e inspecionar um DataFrame

Criando e inspecionando

```
import pandas as pd

jogos = pd.DataFrame({
    "time": ["Tigres", "Águias", "Lobos", "Tigres", "Águias", "Lobos"],
    "rodada": [1, 1, 1, 2, 2, 2],
    "gols": [3, 1, 0, 2, 2, 4],
    "sofridos": [1, 1, 3, 2, 0, 1],
})

print(jogos.head(3))          # primeiras linhas
print(jogos.shape)           # (linhas, colunas)
print(jogos[["rodada", "gols"]].dtypes.astype(str).to_dict()) # tipos
print(jogos["gols"].agg(["min", "max", "mean"]).to_dict())
```

► SAÍDA

```
   time rodada  gols  sofridos
0  Tigres      1     3         1
1  Águias      1     1         1
2  Lobos      1     0         3
(6, 4)
{'rodada': 'int64', 'gols': 'int64'}
{'min': 0.0, 'max': 4.0, 'mean': 2.0}
```

- De arquivo: `pd.read_csv('arquivo.csv')`, `pd.read_excel(...)`.
- Inspeção rápida: `.head()`, `.info()`, `.describe()`, `.shape`, `.columns`, `.dtypes`.
- Para salvar: `df.to_csv('saida.csv', index=False)`.

3.2 Selecionar e filtrar

Colunas: `df['gols']` (uma coluna → Series) ou `df[['time', 'gols']]` (várias → DataFrame). Linhas por condição: uma *máscara booleana* dentro de colchetes. Combine condições com `&` (e), `|` (ou), `~` (não), **sempre com parênteses** em cada condição.

Filtros, isin, &, loc e iloc

```
goleadas = jogos[jogos["gols"] >= 3]
print(goleadas)

so_tigres_e_lobos = jogos[jogos["time"].isin(["Tigres", "Lobos"])]
print(len(so_tigres_e_lobos))

bons = jogos[(jogos["gols"] > 1) & (jogos["sofridos"] <= 1)]
print(bons[["time", "rodada"]])

print(jogos.loc[0, "gols"], jogos.iloc[1, 2]) # por rótulo / por posição
```

► SAÍDA

```

   time rodada  gols  sofridos
0  Tigres      1     3         1
5  Lobos      2     4         1
4
   time rodada
0  Tigres      1
4  Águias      2
5  Lobos      2
3 1

```

CUIDADO Escrever `df[df.a > 1 and df.b < 5]` dá erro. Em pandas use `&|` e parenteses: `df[(df.a > 1) & (df.b < 5)]`.

Quero...	Código
texto que contém	<code>df[df['nome'].str.contains('ana', case=False)]</code>
entre dois valores	<code>df[df['gols'].between(2, 4)]</code>
valores de uma lista	<code>df[df['time'].isin(['Lobos', 'Águias'])]</code>
negar uma condição	<code>df[~(df['gols'] == 0)]</code>

3.3 Colunas novas, ordenar e renomear

Colunas derivadas, `apply`, `sort_values` e `rename`

```

jogos["saldo"] = jogos["gols"] - jogos["sofridos"]          # coluna nova
jogos["resultado"] = jogos["saldo"].apply(
    lambda s: "vitória" if s > 0 else ("empate" if s == 0 else "derrota"))

ordenado = jogos.sort_values(["saldo", "gols"], ascending=[False, False])
print(ordenado[["time", "rodada", "saldo", "resultado"]].to_string(index=False))

renomeado = jogos.rename(columns={"sofridos": "gols_contra"})
print(list(renomeado.columns))

```

► SAÍDA

```

   time rodada  saldo resultado
Lobos      2     3  vitória
Tigres      1     2  vitória
Águias      2     2  vitória
Tigres      2     0  empate
Águias      1     0  empate
Lobos      1    -3  derrota
['time', 'rodada', 'gols', 'gols_contra', 'saldo', 'resultado']

```

DICA Prefira operações entre colunas (`df.a - df.b`) a `apply`: são muito mais rápidas. Use `apply` quando a regra for complexa demais para uma expressão.

3.4 Agrupar e resumir (groupby)

Receita universal: **divida** a tabela em grupos, **aplique** uma agregação em cada um (soma, média, contagem...) e **combine** o resultado.

agg com nomes, value_counts e nunique

```
classificacao = jogos.groupby("time").agg(
    jogos=("rodada", "count"),
    pontos=("saldo", lambda s: (s > 0).sum() * 3 + (s == 0).sum()),
    gols_pro=("gols", "sum"),
    media_saldo=("saldo", "mean"),
).sort_values("pontos", ascending=False)
print(classificacao)

print(jogos["resultado"].value_counts().to_dict()) # contagem por categoria
print(jogos["time"].nunique(), "times diferentes")
```

► SAÍDA

```
jogos  pontos  gols_pro  media_saldo
time
Tigres      2        4        5         1.0
Águias      2        4        3         1.0
Lobos       2        3        4         0.0
{'vitória': 3, 'empate': 2, 'derrota': 1}
3 times diferentes
```

LEMBRE `groupby('coluna')['valor'].sum()` devolve uma **Series** cujo índice são os grupos. Use `.reset_index()` para transformar de volta em tabela comum.

3.5 Limpeza de dados

Dados reais vêm sujos. As ferramentas de sempre:

isna, strip/title, dropna, to_numeric, fillna, drop_duplicates

```
import pandas as pd

sujo = pd.DataFrame({
    "cliente": ["marcos ", "LUCIA", "marcos", None, "Paulo"],
    "idade": ["31", "28", "31", "40", "x"],
    "compra": ["R$ 10,50", "R$ 20,00", "R$ 10,50", "R$ 5,00", None],
})

print(sujo.isna().sum().to_dict()) # nulos por coluna

s = sujo.copy()
s["cliente"] = s["cliente"].str.strip().str.title() # texto padronizado
s = s.dropna(subset=["cliente"]) # sem nome, descarta
s["idade"] = pd.to_numeric(s["idade"], errors="coerce") # 'x' vira NaN
s["idade"] = s["idade"].fillna(s["idade"].median())
s["compra"] = (s["compra"].str.replace("R$ ", "", regex=False)
               .str.replace(",", ".", regex=False).astype(float))
s["compra"] = s["compra"].fillna(0.0)
s = s.drop_duplicates()
print(s.reset_index(drop=True))
```

► SAÍDA

```
{'cliente': 1, 'idade': 0, 'compra': 1}
cliente  idade  compra
0  Marcos   31.0   10.5
1   Lucia   28.0   20.0
2   Paulo   31.0    0.0
```

Problema	Ferramenta
valores vazios (NaN)	<code>isna()</code> · <code>dropna()</code> · <code>fillna(valor)</code>
linhas repetidas	<code>drop_duplicates(subset=[...])</code>
texto sujo	<code>.str.strip()</code> · <code>.str.lower()</code> · <code>.str.title()</code> · <code>.str.replace()</code>
tipo errado	<code>.astype(int)</code> · <code>pd.to_numeric(errors='coerce')</code> · <code>pd.to_datetime()</code>

Problema	Ferramenta
valores impossíveis	<code>df[df['idade'].between(0, 120)]</code>

3.6 Tabela dinâmica (pivot_table)

É o *groupby* em duas dimensões, já com cara de planilha: uma variável nas linhas, outra nas colunas e uma agregação nas células.

pivot_table com totais (margins)

```
import pandas as pd

ingressos = pd.DataFrame({
    "sala": ["1", "1", "2", "2", "1", "2", "2"],
    "dia": ["sex", "sáb", "sex", "sáb", "sáb", "sex", "sáb"],
    "vendidos": [80, 120, 60, 90, 30, 25, 40],
})
tabela = pd.pivot_table(ingressos, values="vendidos", index="sala",
                        columns="dia", aggfunc="sum", fill_value=0,
                        margins=True, margins_name="Total")

print(tabela)
```

► SAÍDA

```
dia    sex  sáb  Total
sala
1         80  150   230
2         85  130   215
Total  165  280   445
```

3.7 Juntar tabelas (merge e concat)

merge junta tabelas por uma chave (como o JOIN do SQL). O parâmetro **how** decide quem fica: *inner* (só quem tem par nos dois lados), *left* (tudo da esquerda), *right*, *outer* (tudo). **concat** empilha tabelas de mesmo formato.

merge inner vs left, e concat

```
import pandas as pd

alunos = pd.DataFrame({"id": [1, 2, 3], "nome": ["Lara", "Caio", "Rui"]})
provas = pd.DataFrame({"aluno_id": [1, 1, 3], "nota": [8.0, 9.0, 5.5]})

inner = alunos.merge(provas, left_on="id", right_on="aluno_id") # só com prova
left = alunos.merge(provas, left_on="id", right_on="aluno_id", how="left")
print(len(inner), len(left))
print(left[["nome", "nota"]])

mais = pd.DataFrame({"id": [4], "nome": ["Eva"]})
print(pd.concat([alunos, mais], ignore_index=True)["nome"].tolist())
```

► SAÍDA

```
3 4
   nome  nota
0  Lara   8.0
1  Lara   9.0
2  Caio  NaN
3  Rui    5.5
['Lara', 'Caio', 'Rui', 'Eva']
```

CUIDADO Depois de um merge *left*, quem não tem par fica com **NaN**. Conte as linhas antes e depois: se o número cresceu, havia chaves repetidas do outro lado (um aluno com 2 provas vira 2 linhas).

Exemplo resolvido — a artilharia do campeonato

ENUNCIADO Com a tabela de jogos abaixo, descubra: (a) o total de gols de cada time, (b) o time com mais gols, (c) a média de gols por jogo de cada time arredondada em 2 casas, (d) quantos jogos tiveram 3 gols ou mais.

Como pensar:

1. **Entender:** uma linha = um time em um jogo.
2. **(a)** `groupby('time')['gols'].sum()`.
3. **(b)** `idxmax()` sobre a Series do passo (a).
4. **(c)** `groupby + mean() + round(2)`. **(d)** filtro e `len()`.

Solução completa

```
import pandas as pd

jogos = pd.DataFrame({
    "time": ["Tigres", "Águias", "Lobos", "Tigres", "Águias", "Lobos", "Tigres"],
    "gols": [3, 1, 0, 2, 2, 4, 1],
})

gols_por_time = jogos.groupby("time")["gols"].sum()
artilheiro = gols_por_time.idxmax()
media = jogos.groupby("time")["gols"].mean().round(2)
jogos_3mais = len(jogos[jogos["gols"] >= 3])

print(gols_por_time.to_dict())
print(artilheiro, media.to_dict(), jogos_3mais)
```

► SAÍDA

```
{'Lobos': 4, 'Tigres': 6, 'Águias': 3}
Tigres {'Lobos': 2.0, 'Tigres': 2.0, 'Águias': 1.5} 2
```

Armadilhas comuns

- ! Usar **and / or** em filtros (precisa de **& e |** com parênteses).
- ! **SettingWithCopyWarning**: ao alterar uma fatia, trabalhe numa cópia (`df.copy()`).
- ! Esquecer que `sum()` ignora NaN e `mean()` também (a média é só dos preenchidos).
- ! Comparar texto com espaços escondidos: padronize com `.str.strip()` antes de agrupar ou juntar.
- ! Esquecer o `reset_index()` depois de um `groupby` e estranhar que o grupo virou índice.

Cola rápida

Quero...	Use...
ver o formato dos dados	<code>df.head()</code> <code>df.info()</code> <code>df.describe()</code>
filtrar linhas	<code>df[(df.a > 1) & (df.b == 'x')]</code>
coluna nova	<code>df['nova'] = df['a'] * df['b']</code>
soma por categoria	<code>df.groupby('cat')['valor'].sum()</code>
maior/menor (rótulo)	<code>serie.idxmax()</code> <code>serie.idxmin()</code>
tabela cruzada	<code>pd.pivot_table(df, values=, index=, columns=, aggfunc=)</code>
juntar tabelas	<code>a.merge(b, on='chave', how='left')</code>
limpar	<code>dropna</code> · <code>fillna</code> · <code>drop_duplicates</code> · <code>astype</code> · <code>str.strip</code>

Exercícios do capítulo 3

Resolva no app (aba **Apostila**) e confira com a correção automática. Antes de digitar, use o rascunho para anotar entrada, saída e passos.

Exercício 7 — Vendas da cafeteria: coluna nova e groupby

[Apostila ► ap_cafeteria_df](#)

O DataFrame "vendas" tem as colunas bebida, qtd e preco.

- 1) Crie a coluna "total" = qtd * preco;
- 2) receita_por_bebida: Series com a soma de "total" por bebida (use groupby);
- 3) bebida_campea: nome da bebida com maior receita.

Ponto de partida (já vem pronto no editor do app):

```
import pandas as pd

vendas = pd.DataFrame({
    "bebida": ["Café", "Cappuccino", "Café", "Chá", "Cappuccino", "Café"],
    "qtd":    [2, 1, 3, 1, 2, 1],
    "preco":  [6.0, 9.5, 6.0, 5.0, 9.5, 6.0],
})

# TODO 1: coluna "total"

# TODO 2: soma de total por bebida
receita_por_bebida = None

# TODO 3: nome da bebida campeã
bebida_campea = ""
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 8 — Limpeza do cadastro de alunos

[Apostila ► ap_limpeza_alunos](#)

O DataFrame "bruto" veio de uma planilha bagunçada. Gere o DataFrame "limpo" seguindo, nesta ordem:

- 1) nomes sem espaços nas pontas e com Iniciais Maiúsculas;
- 2) remova linhas sem nome;
- 3) remova linhas duplicadas;
- 4) converta "idade" para número inteiro;
- 5) nota ausente significa que o aluno faltou: preencha com 0.0.

Dica: use `reset_index(drop=True)` no final.

Ponto de partida (já vem pronto no editor do app):

```
import pandas as pd

bruto = pd.DataFrame({
    "nome": [" ana lima", "BRUNO souza", "Ana Lima ", "carla dias", None],
    "idade": ["20", "21", "20", "19", "22"],
    "nota": [8.0, None, 8.0, 7.5, 6.0],
})

limpo = bruto.copy()
# TODO: aplique os 5 passos em "limpo"
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 9 — Pivot de faltas por turma e mês

Apostila ► [ap_pivot_faltas](#)

O DataFrame "freq" registra faltas por turma e mês.

Crie:

- pivot: tabela com uma linha por turma, uma coluna por mês e a SOMA de faltas (células vazias = 0);
- turma_mais_faltas: nome da turma com o maior total de faltas (somando todos os meses).

Ponto de partida (já vem pronto no editor do app):

```
import pandas as pd

freq = pd.DataFrame({
    "turma": ["A", "A", "B", "B", "A", "B"],
    "mes": ["mar", "abr", "mar", "abr", "abr", "mar"],
    "faltas": [2, 1, 3, 0, 4, 1],
})

# TODO: pivot_table com soma de faltas
pivot = None

# TODO: turma com mais faltas no total
turma_mais_faltas = ""
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 10 — Pacientes e consultas com merge

Apostila ► [ap_merge_pacientes](#)

Há duas tabelas: "pacientes" (id, nome) e "consultas" (paciente_id, valor).

Crie:

- base: junção que mantém TODOS os pacientes, mesmo sem consulta (merge com how="left");
- gasto_por_paciente: Series com o total de "valor" por nome (quem não teve consulta soma 0);

- `sem_consulta`: lista com os nomes de quem não tem nenhuma consulta.

Ponto de partida (já vem pronto no editor do app):

```
import pandas as pd

pacientes = pd.DataFrame({"id": [1, 2, 3], "nome": ["Ana", "Bruno", "Carla"]})
consultas = pd.DataFrame({"paciente_id": [1, 1, 2], "valor": [150.0, 200.0, 180.0]})

# TODO: merge mantendo todos os pacientes
base = None

# TODO: total gasto por nome
gasto_por_paciente = None

# TODO: nomes sem consulta
sem_consulta = []
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

CAPÍTULO 04

SQLite e SQL — banco de dados

Um **banco de dados relacional** guarda dados em tabelas ligadas por chaves. **SQL** é a linguagem para perguntar e alterar esses dados. O **SQLite** é um banco completo dentro de um único arquivo (ou da memória), já incluído no Python: perfeito para estudar e para apps pequenos.

4.1 Conceitos e o módulo sqlite3

- **Tabela:** conjunto de linhas (registros) e colunas (campos).
- **Chave primária (PRIMARY KEY):** identifica cada linha de forma única.
- **Chave estrangeira:** coluna que aponta para a chave primária de outra tabela.
- Tipos do SQLite: INTEGER, REAL, TEXT, BLOB (e NULL para ausência).

Fluxo no Python: **connect** → **execute** (comandos) → **commit** (grava alterações) → **fetchall/fetchone** (lê resultados) → **close**.

Criar tabela, inserir e ler

```
import sqlite3

conn = sqlite3.connect(":memory:") # ou sqlite3.connect("oficina.db")
conn.execute("""
    CREATE TABLE servicos (
        id        INTEGER PRIMARY KEY,
        placa     TEXT NOT NULL,
        tipo      TEXT,
        valor     REAL
    )""")
conn.executemany(
    "INSERT INTO servicos (placa, tipo, valor) VALUES (?, ?, ?)",
    [("ABC1D23", "revisão", 450.0), ("XYZ9K88", "freio", 280.0),
     ("ABC1D23", "óleo", 120.0), ("QWE4R56", "revisão", 520.0)])
conn.commit()

print(conn.execute("SELECT COUNT(*) FROM servicos").fetchone())
for linha in conn.execute("SELECT placa, tipo FROM servicos LIMIT 2"):
    print(linha)
```

► SAÍDA

```
(4,)
('ABC1D23', 'revisão')
('XYZ9K88', 'freio')
```

LEMBRE Sem **commit()** as alterações (INSERT/UPDATE/DELETE) não são gravadas no arquivo. Em bancos em memória isso não aparece, mas em arquivo faz toda a diferença.

4.2 Consultar: SELECT, WHERE, ORDER BY, LIMIT

Cláusula	Função	Exemplo
SELECT	quais colunas	SELECT placa, valor
WHERE	filtra linhas	WHERE valor >= 300
ORDER BY	ordena	ORDER BY valor DESC
LIMIT	limita a quantidade	LIMIT 3
LIKE / IN / BETWEEN	padrão / lista / faixa	WHERE tipo LIKE 'rev%'
IS NULL	testa ausência	WHERE valor IS NULL

WHERE com parâmetros, LIKE, BETWEEN

```
caros = conn.execute(
    "SELECT placa, tipo, valor FROM servicos "
    "WHERE valor >= ? AND tipo != ? ORDER BY valor DESC", (300, "freio")
).fetchall()
print(caros)

print(conn.execute(
    "SELECT tipo FROM servicos WHERE tipo LIKE 'rev%' OR valor BETWEEN 100 AND 150"
).fetchall())
```

► SAÍDA

```
[('QWE4R56', 'revisão', 520.0), ('ABC1D23', 'revisão', 450.0)]
[('revisão',), ('óleo',), ('revisão',)]
```

CUIDADO Nunca monte SQL juntando texto com f-string (`f"... WHERE nome = '{nome}'"`): abre brecha para SQL injection e quebra com aspas. Use `?` e passe os valores numa tupla.

4.3 Alterar dados: UPDATE e DELETE

Reajuste de 10% e remoção

```
conn.execute("UPDATE servicos SET valor = valor * 1.10 WHERE tipo = ?", ("revisão",))
conn.execute("DELETE FROM servicos WHERE valor < ?", (150,))
conn.commit()
print(conn.execute("SELECT tipo, ROUND(valor, 2) FROM servicos ORDER BY id").fetchall())
```

► SAÍDA

```
[('revisão', 495.0), ('freio', 280.0), ('revisão', 572.0)]
```

CUIDADO UPDATE ou DELETE **sem WHERE** atinge a tabela inteira. Escreva primeiro o SELECT com o mesmo WHERE para conferir o que será afetado.

4.4 Agregações: COUNT, SUM, AVG, GROUP BY e HAVING

As funções de agregação resumem várias linhas em uma: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**. Com **GROUP BY** você resume por grupo; com **HAVING** filtra os grupos (WHERE filtra as linhas *antes* de agrupar).

GROUP BY + HAVING

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.executescript("""
CREATE TABLE servicos (placa TEXT, tipo TEXT, valor REAL);
INSERT INTO servicos VALUES
  ('ABC1D23','revisão',450), ('XYZ9K88','freio',280),
  ('ABC1D23','óleo',120),   ('QWE4R56','revisão',520),
  ('ABC1D23','freio',300);
""")
q = """
SELECT placa, COUNT(*) AS qtd, SUM(valor) AS total
FROM servicos
GROUP BY placa
HAVING COUNT(*) >= 2
ORDER BY total DESC
"""
print(conn.execute(q).fetchall())
print(conn.execute("SELECT tipo, AVG(valor) FROM servicos GROUP BY tipo").fetchall())
```

► SAÍDA

```
[('ABC1D23', 3, 870.0)]
[('freio', 290.0), ('revisão', 485.0), ('óleo', 120.0)]
```

LEMBRE Ordem de escrita: **SELECT** → **FROM** → **JOIN** → **WHERE** → **GROUP BY** → **HAVING** → **ORDER BY** → **LIMIT**.

4.5 Juntar tabelas: JOIN

Dados bem organizados ficam em tabelas separadas (um cliente, vários pedidos). **JOIN** as combina pela chave. **INNER JOIN** traz só quem tem par; **LEFT JOIN** traz tudo da tabela da esquerda, com NULL onde não há par.

INNER JOIN vs LEFT JOIN (Sônia só aparece no LEFT)

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.executescript("""
CREATE TABLE donos (id INTEGER PRIMARY KEY, nome TEXT);
CREATE TABLE carros (id INTEGER PRIMARY KEY, dono_id INTEGER, modelo TEXT);
INSERT INTO donos VALUES (1,'Marta'), (2,'Otávio'), (3,'Sônia');
INSERT INTO carros (dono_id, modelo) VALUES (1,'Gol'), (1,'Onix'), (2,'Argo');
""")
print(conn.execute("""
SELECT d.nome, c.modelo FROM donos d
INNER JOIN carros c ON c.dono_id = d.id ORDER BY d.nome""").fetchall())
print(conn.execute("""
SELECT d.nome, COUNT(c.id) AS carros FROM donos d
LEFT JOIN carros c ON c.dono_id = d.id
GROUP BY d.id ORDER BY d.nome""").fetchall())
```

► SAÍDA

```
[('Marta', 'Gol'), ('Marta', 'Onix'), ('Otávio', 'Argo')]
[('Marta', 2), ('Otávio', 1), ('Sônia', 0)]
```

DICA Use **COUNT(c.id)** (coluna da tabela da direita) e não **COUNT(*)** no **LEFT JOIN**: assim quem não tem par conta 0, e não 1.

4.6 Pandas + SQL

to_sql e read_sql

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(":memory:")
df = pd.DataFrame({"peca": ["filtro", "pastilha", "filtro"], "qtd": [4, 2, 6]})
df.to_sql("consumo", conn, index=False, if_exists="replace") # DataFrame -> tabela

resumo = pd.read_sql(
    "SELECT peca, SUM(qtd) AS total FROM consumo GROUP BY peca", conn) # SQL -> DataFrame
print(resumo)
```

► SAÍDA

```
   peca  total
0  filtro     10
1 pastilha     2
```

LEMBRE Regra prática: **filtrar e agregar** volumes grandes no SQL; **explorar e transformar** o resultado no pandas.

Exemplo resolvido — ranking de peças da oficina

ENUNCIADO Dada a tabela pecas(nome, categoria, preco, estoque), liste, por categoria, o valor total em estoque (preco × estoque) mostrando apenas categorias cujo total passe de R\$ 1.000, da maior para a menor.

Como pensar:

1. **Pergunta** → **SQL**: 'por categoria' = GROUP BY categoria.
2. **Valor em estoque** = SUM(preco * estoque).
3. **'só as que passam de 1000'** é filtro de GRUPO → HAVING (não WHERE).
4. **Ordem** = ORDER BY total DESC.

Solução completa

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.executescript("""
CREATE TABLE pecas (nome TEXT, categoria TEXT, preco REAL, estoque INTEGER);
INSERT INTO pecas VALUES
('Filtro de ar', 'motor', 40, 30), ('Vela', 'motor', 25, 100),
('Pastilha', 'freio', 120, 12), ('Disco', 'freio', 200, 8),
('Lâmpada', 'elétrica', 12, 40);
""")
print(conn.execute("""
SELECT categoria, SUM(preco * estoque) AS total
FROM pecas
GROUP BY categoria
HAVING SUM(preco * estoque) > 1000
ORDER BY total DESC""").fetchall())
```

► SAÍDA

```
[('motor', 3700.0), ('freio', 3040.0)]
```

Armadilhas comuns

- ! Esquecer o **commit()** depois de INSERT/UPDATE/DELETE em banco de arquivo.
- ! Formatar SQL com f-string em vez de usar **?** com parâmetros.
- ! Passar um valor só como parâmetro sem tupla: `execute(sql, (valor,))` — repare na vírgula.
- ! Usar **WHERE** para filtrar resultado de agregação (o certo é HAVING).

! No LEFT JOIN, filtrar a tabela da direita no WHERE transforma o LEFT em INNER sem querer; coloque a condição no ON.

Cola rápida

Quero...	SQL
criar tabela	CREATE TABLE t (id INTEGER PRIMARY KEY, nome TEXT)
inserir (seguro)	INSERT INTO t (nome) VALUES (?)
filtrar e ordenar	SELECT ... WHERE ... ORDER BY col DESC LIMIT n
total por grupo	SELECT g, SUM(v) FROM t GROUP BY g
filtrar grupos	... GROUP BY g HAVING SUM(v) > 100
juntar tabelas	FROM a JOIN b ON b.a_id = a.id (LEFT JOIN p/ manter tudo de a)
alterar / apagar	UPDATE t SET c = ? WHERE id = ? · DELETE FROM t WHERE id = ?

Exercícios do capítulo 4

Resolva no app (aba **Apostila**) e confira com a correção automática. Antes de digitar, use o rascunho para anotar entrada, saída e passos.

Exercício 11 — Catálogo de filmes: CREATE, INSERT e SELECT

[Apostila > ap_filmes_sql](#)

A conexão "conn" (banco em memória) já está aberta e a lista "catalogo" tem (titulo, nota).

- 1) Crie a tabela filmes(id INTEGER PRIMARY KEY, titulo TEXT, nota REAL);
- 2) Insira todos os itens de catalogo com executemany e parâmetros "?";
- 3) bem_avalidados: lista de títulos com nota >= 8, da maior para a menor nota (consulta SQL com ORDER BY).

Ponto de partida (já vem pronto no editor do app):

```
import sqlite3

conn = sqlite3.connect(":memory:")
catalogo = [
    ("Cidade de Deus", 8.6),
    ("Central do Brasil", 8.0),
    ("Tropa de Elite", 7.1),
    ("O Auto da Compadecida", 8.8),
]

# TODO 1: CREATE TABLE filmes (...)

# TODO 2: INSERT com executemany e "?"

# TODO 3: títulos com nota >= 8, maior nota primeiro
bem_avalidados = []
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 12 — JOIN, GROUP BY e HAVING em pedidos de restaurante

[Apostila > ap_join_pedidos](#)

As tabelas clientes(id, nome) e pedidos(id, cliente_id, valor) já estão criadas e preenchidas.

Escreva na variável "consulta" um SELECT que devolva, para cada cliente, três colunas NESTA ORDEM: nome, total gasto (SUM do valor) e quantidade de pedidos (COUNT); mostrando só clientes cujo total seja maior que 100, do maior total para o menor.

Ponto de partida (já vem pronto no editor do app):

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.executescript("""
CREATE TABLE clientes (id INTEGER PRIMARY KEY, nome TEXT);
CREATE TABLE pedidos (id INTEGER PRIMARY KEY, cliente_id INTEGER, valor REAL);
INSERT INTO clientes VALUES (1, 'Ana'), (2, 'Bruno'), (3, 'Carla');
INSERT INTO pedidos (cliente_id, valor) VALUES (1, 60), (1, 70), (2, 40), (3, 200);
""")

# TODO: escreva o SELECT (nome, total, qtd) com JOIN + GROUP BY + HAVING + ORDER BY
consulta = ""

resultado = conn.execute(consulta).fetchall() if consulta else []
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 13 — Integração Pandas + SQLite (to_sql e read_sql)

[Apostila > ap_pandas_sql](#)

O DataFrame "df" tem vendas de uma loja virtual.

- 1) Grave-o no banco com df.to_sql("vendas", conn, index=False, if_exists="replace");
- 2) Em df_sql, leia com pd.read_sql o total por categoria (colunas: categoria, total), do maior para o menor total.

Ponto de partida (já vem pronto no editor do app):

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(":memory:")
df = pd.DataFrame({
    "categoria": ["Livros", "Games", "Livros", "Música"],
    "valor": [50.0, 200.0, 30.0, 20.0],
})

# TODO 1: df.to_sql(...)

# TODO 2: pd.read_sql("SELECT ...", conn)
df_sql = None
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

CAPÍTULO 05

Tkinter — interfaces gráficas

Tkinter vem instalado com o Python e cria janelas com botões, campos de texto e listas. A lógica é sempre a mesma: **criar a janela**, **criar widgets**, **posicioná-los**, **ligar eventos a funções** e chamar `mainloop()`.

5.1 Estrutura mínima de uma janela

```
import tkinter as tk

janela = tk.Tk()                # janela principal
janela.title("Meu app")
janela.geometry("300x150")      # largura x altura

rotulo = tk.Label(janela, text="Olá, mundo!")
rotulo.pack(pady=10)           # posiciona (empilha) na janela

janela.mainloop()              # mantém a janela aberta e atenta a eventos
```

LEMBRE O código depois de `mainloop()` só roda quando a janela fecha. Tudo o que o app faz acontece dentro de *callbacks* (funções chamadas por eventos).

CUIDADO Tkinter precisa de uma tela. Ele **não abre** em navegador, em servidor sem interface nem na aba Apostila do app: lá validamos a estrutura. Para ver a janela de verdade, rode o arquivo `.py` no seu computador.

5.2 Widgets mais usados

Widget	Para quê	Pontos-chave
Label	mostrar texto	<code>label.config(text='novo texto')</code>
Button	disparar uma ação	<code>Button(janela, text='OK', command=funcao)</code>
Entry	uma linha de texto	<code>entrada.get()</code> · <code>entrada.delete(0, tk.END)</code>
Listbox	lista de itens	<code>lista.insert(tk.END, item)</code> · <code>lista.curselection()</code>
Frame	agrupar widgets	<code>Frame(janela).pack()</code> (container)
messagebox	avisos e perguntas	<code>from tkinter import messagebox</code>

5.3 Posicionamento: pack, grid e place

- **pack()**: empilha um widget embaixo do outro (rápido para telas simples). Opções: `side`, `padx`, `pady`, `fill`, `expand`.
- **grid(row=, column=)**: posiciona em linhas e colunas, ideal para formulários. Não misture `pack` e `grid` no mesmo container.
- **place(x=, y=)**: posição absoluta em pixels (evite; não se adapta ao redimensionar).

```
# formulário com grid
tk.Label(janela, text="Nome:").grid(row=0, column=0, sticky="e", padx=5, pady=5)
nome = tk.Entry(janela)
nome.grid(row=0, column=1, padx=5, pady=5)
```

```
tk.Label(janela, text="Idade:").grid(row=1, column=0, sticky="e", padx=5)
idade = tk.Entry(janela)
idade.grid(row=1, column=1, padx=5)
```

5.4 Eventos e callbacks

O botão recebe a função em **command=**, **sem parênteses**: quem a chama é o Tkinter, no clique. Para outros eventos (tecla Enter, clique duplo) use **widget.bind('<Return>', funcao)** — nesse caso a função recebe o parâmetro *evento*.

```
import tkinter as tk
from tkinter import messagebox

def converter():
    try:
        reais = float(entrada.get().replace(",", "."))
    except ValueError:
        messagebox.showerror("Erro", "Digite um número válido")
    return
    resultado.config(text=f"US$ {reais / 5.0:.2f}") # cotação fixa de exemplo

janela = tk.Tk()
janela.title("Conversor")
entrada = tk.Entry(janela)
entrada.pack(padx=10, pady=5)
entrada.bind("<Return>", lambda evento: converter()) # Enter também converte
tk.Button(janela, text="Converter R$ → US$", command=converter).pack()
resultado = tk.Label(janela, text="")
resultado.pack(pady=8)
janela.mainloop()
```

DICA Sempre valide o que vem do **Entry**: ele devolve texto. Converta com `float()/int()` dentro de `try/except` e avise o usuário em vez de deixar o app quebrar.

5.5 Organizando o app em uma classe

Variáveis globais viram bagunça rápido. Herdando de **tk.Tk** (ou usando um `Frame`), widgets e dados viram atributos de *self* e cada ação, um método.

```
import tkinter as tk

class AppContador(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Contador")
        self.total = 0
        self.rotulo = tk.Label(self, text="0", font=("Arial", 24))
        self.rotulo.pack(padx=30, pady=10)
        tk.Button(self, text="+1", command=self.somar).pack(pady=(0, 10))

    def somar(self):
        self.total += 1
        self.rotulo.config(text=str(self.total))

if __name__ == "__main__":
    AppContador().mainloop()
```

Exemplo resolvido — checklist de viagem

ENUNCIADO Uma janela com um `Entry`, o botão 'Adicionar' e um `Listbox`. Ao adicionar, o item vai para a lista e o campo é limpo; um segundo botão 'Remover selecionado' apaga o item escolhido.

Como pensar:

1. **Widgets:** `Entry`, 2 `Buttons`, `Listbox`.

2. **Callback adicionar:** `get()` → `insert(END, ...)` → `delete(0, END)`.
3. **Callback remover:** `curselection()` devolve as posições selecionadas; `delete(posição)`.

Solução completa (rode no seu computador)

```
import tkinter as tk

def adicionar():
    item = entrada.get().strip()
    if item:
        lista.insert(tk.END, item)
        entrada.delete(0, tk.END)

def remover():
    for posicao in reversed(lista.curselection()):
        lista.delete(posicao)

janela = tk.Tk()
janela.title("Checklist")
entrada = tk.Entry(janela, width=28)
entrada.pack(padx=10, pady=5)
tk.Button(janela, text="Adicionar", command=adicionar).pack()
lista = tk.Listbox(janela, width=32, height=8)
lista.pack(padx=10, pady=8)
tk.Button(janela, text="Remover selecionado", command=remover).pack(pady=(0, 8))
janela.mainloop()
```

Armadilhas comuns

- ! **`command=funcao()`** com parênteses executa a função na hora de criar o botão. Use **`command=funcao`** (ou *lambda: funcao(x)* para passar argumentos).
- ! Misturar **pack** e **grid** no mesmo container trava a janela.
- ! Fazer algo demorado dentro de um callback congela a janela (ela só responde entre callbacks).
- ! Esquecer de guardar o widget numa variável antes de chamar **`.pack()`**: `x = tk.Label(...).pack()` faz x valer None.

Cola rápida

Quero...	Tkinter
criar janela	<code>janela = tk.Tk(); janela.title('...'); janela.mainloop()</code>
ler / limpar campo	<code>entrada.get()</code> <code>entrada.delete(0, tk.END)</code>
trocar texto	<code>label.config(text='...')</code>
adicionar na lista	<code>lista.insert(tk.END, texto)</code>
ligar botão a função	<code>tk.Button(janela, text='OK', command=funcao)</code>
avisar o usuário	<code>messagebox.showinfo('Título', 'Mensagem')</code>

Exercícios do capítulo 5

Resolva no app (aba **Apostila**) e confira com a correção automática. Antes de digitar, use o rascunho para anotar entrada, saída e passos.

Exercício 14 — Contador de cliques

[Apostila > apTkContador](#)

Crie uma janela Tkinter com um Label que mostra "Cliques: 0" e um Button "Clicar". A cada clique, o número aumenta de 1 e o Label é atualizado.

☐☐ O navegador não abre janelas Tkinter. Aqui validamos a ESTRUTURA do código; copie e rode no seu

computador para ver a janela de verdade.

Ponto de partida (já vem pronto no editor do app):

```
import tkinter as tk

contagem = 0

def clicar():
    # TODO: aumente a contagem e atualize o label
    pass

janela = tk.Tk()
janela.title("Contador")

# TODO: crie o Label "Cliques: 0" e o Button "Clicar" (command=clicar) e use pack()

# janela.mainloop() # deixe comentado aqui; rode localmente
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Exercício 15 — Lista de tarefas com Entry e Listbox

Apostila ► [ap_tk_tarefas](#)

Monte uma janela com: um Entry para digitar uma tarefa, um Button "Adicionar" e um Listbox com as tarefas. Ao clicar, o texto do Entry vai para o fim do Listbox e o Entry é limpo.

□□ Validação estrutural (o navegador não abre janelas Tkinter). Rode no seu computador para ver funcionando.

Ponto de partida (já vem pronto no editor do app):

```
import tkinter as tk

def adicionar():
    # TODO: pegue o texto do Entry, insira no Listbox e limpe o Entry
    pass

janela = tk.Tk()
janela.title("Tarefas")

# TODO: Entry, Button (command=adicionar) e Listbox, todos com pack()

# janela.mainloop()
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

CAPÍTULO 06

Projeto integrador

Aqui as peças se encontram: **classes** para as regras, **SQLite** para guardar, **pandas** para analisar e, opcionalmente, **Tkinter** para a tela. O segredo não é código novo, e sim *onde cada coisa mora*.

6.1 Arquitetura em camadas

Camada	Responsabilidade	Ferramenta
Dados	guardar e buscar registros	sqlite3 (CREATE, INSERT, SELECT)
Regras	o que pode e o que não pode	classes + exceções
Análise	relatórios e números	pandas (groupby, pivot)
Interface	falar com o usuário	print/input ou Tkinter

LEMBRE Uma camada só conversa com a vizinha. A interface não escreve SQL; o banco não conhece a interface. Assim você troca Tkinter por uma página web depois sem mexer nas regras.

6.2 Roteiro para qualquer projeto novo

1. **Descreva o problema** em 3 frases (quem usa, o que registra, que relatório quer).
2. **Liste as entidades** (substantivos): ex.: Aluno, Turma, Presença.
3. **Desenhe as tabelas**: uma por entidade, com chave primária e as chaves estrangeiras que ligam uma à outra.
4. **Crie as classes**: uma por entidade, com as regras (validações) como métodos.
5. **Escreva a camada de dados**: métodos adicionar / buscar / listar.
6. **Faça os relatórios** com SQL (agregações) e pandas (apresentação).
7. **Só então** monte a interface. Teste cada camada isolada antes de ligar tudo.

6.3 Esqueleto completo (estúdio de música)

Um agendamento de salas de ensaio: classe com a regra, banco guardando, pandas resumindo. Leia de cima para baixo: cada bloco é uma camada.

Classe + SQLite + pandas trabalhando juntos

```
import sqlite3
import pandas as pd

class Estudo:
    """Camada de dados + regras de agendamento."""

    def __init__(self, caminho=":memory:"):
        self.conn = sqlite3.connect(caminho)
        self.conn.execute("""CREATE TABLE IF NOT EXISTS ensaios (
            id INTEGER PRIMARY KEY, banda TEXT, sala INTEGER,
            horas INTEGER, valor_hora REAL)""")

    def agendar(self, banda, sala, horas, valor_hora=60.0):
        if horas <= 0:
            raise ValueError("horas deve ser positivo")
        self.conn.execute(
            "INSERT INTO ensaios (banda, sala, horas, valor_hora) VALUES (?, ?, ?, ?)",
            (banda, sala, horas, valor_hora))
        self.conn.commit()

    def relatorio(self):
        """Camada de análise: SQL entrega a tabela, pandas organiza."""
        df = pd.read_sql(
            "SELECT banda, horas * valor_hora AS valor FROM ensaios", self.conn)
        total = df.groupby("banda", as_index=False)["valor"].sum()
        return total.sort_values("valor", ascending=False)

est = Estudo()
est.agendar("Trio Maré", 1, 2)
est.agendar("Banda Aurora", 2, 3, valor_hora=80.0)
est.agendar("Trio Maré", 1, 1)
print(est.relatorio().to_string(index=False))
try:
    est.agendar("Solo", 1, 0)
except ValueError as erro:
    print("recusado:", erro)
```

► SAÍDA

```
      banda  valor
Banda Aurora  240.0
      Trio Maré  180.0
recusado: horas deve ser positivo
```

DICA Desafio para praticar: implemente a regra de conflito de horário (duas bandas não podem ocupar a mesma sala no mesmo horário). Comece escrevendo o SELECT que detecta o conflito, depois use o resultado dentro de agendar().

Armadilhas comuns

- ! Começar pela interface: sem as regras e os dados funcionando, a tela só esconde os erros.
- ! Misturar SQL espalhado pelo programa todo: concentre numa classe/arquivo de dados.
- ! Guardar dados calculados no banco (ex.: total) em vez de calcular na consulta; eles ficam desatualizados.
- ! Não testar com dados vazios (sem reservas, sem alunos): é aí que SUM() devolve NULL.

Cola rápida

Quero...	Faça...
guardar dados	tabela + classe de acesso com adicionar/listar
proteger uma regra	método da classe que levanta ValueError
relatório rápido	SQL agrega → pandas apresenta
tela	Tkinter chamando os métodos da classe (nunca SQL direto)
testar	rode a classe sozinha com dados de exemplo antes da tela

Exercícios do capítulo 6

Resolva no app (aba **Apostila**) e confira com a correção automática. Antes de digitar, use o rascunho para anotar entrada, saída e passos.

Exercício 16 — Sistema de reservas de hotel (POO + SQLite + pandas)

[Apostila](#) ► [ap_reservas_hotel](#)

Crie a classe ReservaDB que usa um banco SQLite em memória:

- `__init__`: abre a conexão e cria a tabela reservas(id INTEGER PRIMARY KEY, hospede TEXT, noites INTEGER, diaria REAL);
- `adicionar(hospede, noites, diaria)`: insere uma reserva;
- `listar()`: devolve um DataFrame com todas as reservas (`pd.read_sql`);
- `faturamento()`: soma de noites * diaria feita NO SQL (0.0 se não houver reservas);
- `maior_reserva()`: nome do hóspede com maior valor (noites * diaria).

Ponto de partida (já vem pronto no editor do app):

```
import sqlite3
import pandas as pd

class ReservaDB:
    # TODO: __init__, adicionar, listar, faturamento, maior_reserva
    pass

# Depois de implementar, teste assim:
# hotel = ReservaDB()
# hotel.adicionar("Marcos", 3, 200.0)
# print(hotel.listar())
```

Rascunho — escreva a entrada, a saída esperada e os passos antes de programar

Apêndice A — Método para resolver qualquer exercício

1. **Leia o enunciado duas vezes** e sublinhe o que ele PEDE: nomes de variáveis, de funções, formato do resultado.
2. **Anote a entrada e a saída** em uma linha cada: 'recebo uma lista de dicts → devolvo um número'.
3. **Resolva um exemplo à mão** com os dados fornecidos. O resultado é o seu teste.
4. **Escreva a versão mais simples** que funcione (pode ser com for). Funcionar vem antes de ficar bonito.
5. **Rode e compare** com o resultado à mão. Divergiu? Imprima os passos intermediários com print().
6. **Só então melhore**: troque o for por comprehension, nomeie melhor, remova repetição.
7. **Confira os casos de borda**: lista vazia, valor zero, texto com espaços, letra maiúscula.

Apêndice B — Mensagens de erro e o que fazer

Mensagem	O que olhar primeiro
IndentationError: unexpected indent	linha com espaços a mais/menos que o bloco
SyntaxError: invalid syntax	falta ':' ou parêntese; confira a linha de cima também
NameError: name 'x' is not defined	escreveu o nome diferente ou usou antes de criar
KeyError: 'preco'	chave/coluna inexistente (acentos e maiúsculas contam)
TypeError: unsupported operand	somando/multiplicando tipos incompatíveis (texto × número)
AttributeError: 'list' has no attribute ...	método não existe nesse tipo de objeto
ValueError: invalid literal for int()	texto que não é número (ex.: 'abc' ou '3,5')
sqlite3.OperationalError: no such table	tabela não criada nesta conexão (ou nome errado)
sqlite3.ProgrammingError: Incorrect number of bindings	quantidade de ? diferente da de valores

Gabarito comentado

ANTES DE LER Só abra esta parte depois de tentar! Se o seu código passou na correção automática mas é diferente do abaixo, está tudo bem: compare as *ideias* (lista de pontos ao final de cada solução).

Capítulo 1 — Python básico

Exercício 1 — Acervo da biblioteca de bairro

```
livros = [
    {"titulo": "Dom Casmurro", "paginas": 256, "exemplares": 3},
    {"titulo": "O Cortiço", "paginas": 320, "exemplares": 1},
    {"titulo": "Memórias Póstumas", "paginas": 288, "exemplares": 2},
    {"titulo": "Iracema", "paginas": 176, "exemplares": 1},
]

total_paginas = sum(l["paginas"] * l["exemplares"] for l in livros)
livro_mais_longo = max(livros, key=lambda l: l["paginas"])[ "titulo" ]
titulos_raros = [l["titulo"] for l in livros if l["exemplares"] == 1]
```

- › `sum()` com expressão geradora no lugar do `for` + acumulador
- › `max(..., key=...)` devolve o item inteiro; depois pegamos só o título
- › List comprehension com `if` filtra e transforma em uma única linha

Exercício 2 — Validador de matrícula escolar

```
def matricula_valida(texto):
    partes = texto.strip().upper().split("-")
    if len(partes) != 3:
        return False
    prefixo, ano, numero = partes
    return (prefixo == "MAT"
            and len(ano) == 4 and ano.isdigit()
            and len(numero) == 4 and numero.isdigit())
```

- › `strip()` + `upper()` normalizam a entrada antes de validar (uma regra só, sem duplicar casos)
- › `split('-')` + desempacotamento deixa claro o que é cada pedaço
- › `isdigit()` evita converter para `int` só para testar se é número

Exercício 3 — Boletim escolar com funções e f-strings

```
def situacao(notas):
    media = round(sum(notas) / len(notas), 1)
    if media >= 7:
        return (media, "Aprovado")
    if media >= 5:
        return (media, "Recuperação")
    return (media, "Reprovado")

alunos = {
    "Ana": [8, 9, 7.5],
    "Bruno": [5, 6, 4],
    "Carla": [3, 4, 2],
}

boletim = {nome: situacao(notas)[1] for nome, notas in alunos.items()}
```

- › Retornos antecipados (`return` dentro do `if`) dispensam `else/elif` aninhados
- › Dict comprehension monta o resultado direto, sem criar o dict vazio e preencher no loop
- › A função faz uma coisa só (classificar); quem monta o boletim é outro trecho

Capítulo 2 — Programação orientada a objetos

Exercício 4 — Classe Livro com empréstimo e devolução

```
class Livro:
    def __init__(self, titulo, exemplares):
        self.titulo = titulo
        self.exemplares = exemplares

    def emprestar(self):
        if self.exemplares == 0:
            raise ValueError("sem exemplares")
        self.exemplares -= 1

    def devolver(self):
        self.exemplares += 1

    def __str__(self):
        return f"{self.titulo} ({self.exemplares} disponíveis)"
```

- › A regra de negócio (não emprestar sem estoque) fica DENTRO da classe, não espalhada pelo programa
- › `raise ValueError` dá um erro claro; quem chama decide como tratar com `try/except`
- › `__str__` faz `print(livro)` ficar legível sem código extra

Exercício 5 — Planos de academia com herança

```
class Plano:
    def __init__(self, aluno, base):
        self.aluno = aluno
        self.base = base

    def mensalidade(self):
        return self.base

class PlanoAnual(Plano):
    def mensalidade(self):
        return super().mensalidade() * 0.8

class PlanoPersonal(Plano):
    def mensalidade(self):
        return super().mensalidade() + 150

planos = [Plano("Rita", 120), PlanoAnual("Caio", 120), PlanoPersonal("Lia", 120)]

total_mensal = sum(p.mensalidade() for p in planos)
```

- › `super().mensalidade()` reaproveita a regra da classe-mãe em vez de copiar `self.base`
- › Polimorfismo: o loop chama `p.mensalidade()` sem saber qual é o tipo de plano
- › Um novo tipo de plano é só mais uma subclasse; o cálculo do total não muda

Exercício 6 — Carteirainha de cafeteria (encapsulamento)

```
class Carteirainha:
    def __init__(self, dono, saldo=0):
        self.dono = dono
        self._saldo = saldo

    @property
    def saldo(self):
        return self._saldo

    def depositar(self, valor):
        if valor <= 0:
            raise ValueError("valor deve ser positivo")
        self._saldo += valor

    def pagar(self, valor):
        if valor > self._saldo:
            raise ValueError("saldo insuficiente")
        self._saldo -= valor
```

- › Property sem setter impede alterações diretas: o saldo só muda por depositar/pagar
- › Validar ANTES de alterar o estado garante que uma operação recusada não deixa o objeto pela metade
- › O underscore (_saldo) é a convenção de 'uso interno da classe'

Capítulo 3 — Pandas — análise de dados

Exercício 7 — Vendas da cafeteria: coluna nova e groupby

```
import pandas as pd

vendas = pd.DataFrame({
    "bebida": ["Café", "Cappuccino", "Café", "Chá", "Cappuccino", "Café"],
    "qtd": [2, 1, 3, 1, 2, 1],
    "preco": [6.0, 9.5, 6.0, 5.0, 9.5, 6.0],
})

vendas["total"] = vendas["qtd"] * vendas["preco"]
receita_por_bebida = vendas.groupby("bebida")["total"].sum()
bebida_campea = receita_por_bebida.idxmax()
```

- › Operações vetorizadas (coluna * coluna) dispensam loops linha a linha
- › groupby(...)[coluna].sum() é o padrão 'agrupe e some'
- › idxmax() devolve o RÓTULO do maior valor (max() devolveria o valor)

Exercício 8 — Limpeza do cadastro de alunos

```
import pandas as pd

bruto = pd.DataFrame({
    "nome": [" ana lima", "BRUNO souza", "Ana Lima ", "carla dias", None],
    "idade": ["20", "21", "20", "19", "22"],
    "nota": [8.0, None, 8.0, 7.5, 6.0],
})

limpo = bruto.copy()
limpo["nome"] = limpo["nome"].str.strip().str.title()
limpo = limpo.dropna(subset=["nome"])
limpo = limpo.drop_duplicates()
limpo["idade"] = limpo["idade"].astype(int)
limpo["nota"] = limpo["nota"].fillna(0.0)
limpo = limpo.reset_index(drop=True)
```

- › A ordem importa: normalizar o nome ANTES de procurar duplicadas revela 'ana lima' = 'Ana Lima '
- › dropna(subset=[...]) só olha as colunas que importam

- › Sempre trabalhe numa cópia (`bruto.copy()`) para poder comparar antes/depois

Exercício 9 — Pivot de faltas por turma e mês

```
import pandas as pd

freq = pd.DataFrame({
    "turma": ["A", "A", "B", "B", "A", "B"],
    "mes": ["mar", "abr", "mar", "abr", "abr", "mar"],
    "faltas": [2, 1, 3, 0, 4, 1],
})

pivot = pd.pivot_table(freq, values="faltas", index="turma",
                        columns="mes", aggfunc="sum", fill_value=0)
turma_mais_faltas = pivot.sum(axis=1).idxmax()
```

- › `pivot_table` = `groupby` em duas dimensões, já formatado como planilha
- › `fill_value=0` troca as células sem dados por 0
- › `sum(axis=1)` soma ao longo das colunas (por linha); `axis=0` somaria por coluna

Exercício 10 — Pacientes e consultas com merge

```
import pandas as pd

pacientes = pd.DataFrame({"id": [1, 2, 3], "nome": ["Ana", "Bruno", "Carla"]})
consultas = pd.DataFrame({"paciente_id": [1, 1, 2], "valor": [150.0, 200.0, 180.0]})

base = pacientes.merge(consultas, left_on="id", right_on="paciente_id", how="left")
gasto_por_paciente = base.groupby("nome")["valor"].sum()
sem_consulta = base[base["valor"].isna()]["nome"].tolist()
```

- › `how='left'` preserva a tabela da esquerda inteira (é o LEFT JOIN do SQL)
- › Nomes de chave diferentes nas tabelas? `left_on/right_on` resolvem
- › `isna()` acha exatamente quem ficou sem par na junção

Capítulo 4 — SQLite e SQL — banco de dados

Exercício 11 — Catálogo de filmes: CREATE, INSERT e SELECT

```
import sqlite3

conn = sqlite3.connect(":memory:")
catalogo = [
    ("Cidade de Deus", 8.6),
    ("Central do Brasil", 8.0),
    ("Tropa de Elite", 7.1),
    ("O Auto da Compadecida", 8.8),
]

conn.execute("""CREATE TABLE filmes (
    id INTEGER PRIMARY KEY,
    titulo TEXT,
    nota REAL)""")
conn.executemany("INSERT INTO filmes (titulo, nota) VALUES (?, ?)", catalogo)
conn.commit()

linhas = conn.execute(
    "SELECT titulo FROM filmes WHERE nota >= ? ORDER BY nota DESC", (8,))
bem_avalizados = [titulo for (titulo,) in linhas]
```

- › Parâmetros '?' evitam SQL injection e problemas com aspas nos textos
- › `executemany` insere a lista toda de uma vez

- › fetchall() devolve tuplas; a comprehension com (titulo,) desembrulha cada uma

Exercício 12 — JOIN, GROUP BY e HAVING em pedidos de restaurante

```
import sqlite3

conn = sqlite3.connect(":memory:")
conn.executescript("""
CREATE TABLE clientes (id INTEGER PRIMARY KEY, nome TEXT);
CREATE TABLE pedidos (id INTEGER PRIMARY KEY, cliente_id INTEGER, valor REAL);
INSERT INTO clientes VALUES (1, 'Ana'), (2, 'Bruno'), (3, 'Carla');
INSERT INTO pedidos (cliente_id, valor) VALUES (1, 60), (1, 70), (2, 40), (3, 200);
""")

consulta = """
SELECT c.nome, SUM(p.valor) AS total, COUNT(*) AS qtd
FROM clientes c
JOIN pedidos p ON p.cliente_id = c.id
GROUP BY c.id, c.nome
HAVING SUM(p.valor) > 100
ORDER BY total DESC
"""

resultado = conn.execute(consulta).fetchall() if consulta else []
```

- › WHERE filtra LINHAS antes de agrupar; HAVING filtra GRUPOS depois de agregar
- › Apelidos (c, p) deixam o JOIN curto e legível
- › O banco faz a soma e a contagem; o Python só recebe o resultado pronto

Exercício 13 — Integração Pandas + SQLite (to_sql e read_sql)

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(":memory:")
df = pd.DataFrame({
    "categoria": ["Livros", "Games", "Livros", "Música"],
    "valor": [50.0, 200.0, 30.0, 20.0],
})

df.to_sql("vendas", conn, index=False, if_exists="replace")
df_sql = pd.read_sql(
    "SELECT categoria, SUM(valor) AS total FROM vendas "
    "GROUP BY categoria ORDER BY total DESC", conn)
```

- › to_sql transforma um DataFrame em tabela sem escrever CREATE/INSERT
- › read_sql devolve um DataFrame pronto para continuar a análise no pandas
- › index=False evita gravar o índice do DataFrame como uma coluna extra

Capítulo 5 — Tkinter — interfaces gráficas

Exercício 14 — Contador de cliques

```
import tkinter as tk

contagem = 0

def clicar():
    global contagem
    contagem += 1
    label.config(text=f"Cliques: {contagem}")

janela = tk.Tk()
janela.title("Contador")

label = tk.Label(janela, text="Cliques: 0")
label.pack(padx=20, pady=10)
tk.Button(janela, text="Clicar", command=clicar).pack(pady=(0, 10))

janela.mainloop()
```

- › global é necessário para REATRIBUIR uma variável de fora da função
- › Em apps maiores, prefira uma classe com self.contagem no lugar de variável global
- › command=clicar passa a função SEM parênteses (quem chama é o Tkinter, no clique)

Exercício 15 — Lista de tarefas com Entry e Listbox

```
import tkinter as tk

def adicionar():
    texto = entrada.get().strip()
    if texto:
        lista.insert(tk.END, texto)
        entrada.delete(0, tk.END)

janela = tk.Tk()
janela.title("Tarefas")

entrada = tk.Entry(janela, width=30)
entrada.pack(padx=10, pady=(10, 5))
tk.Button(janela, text="Adicionar", command=adicionar).pack()
lista = tk.Listbox(janela, width=40, height=8)
lista.pack(padx=10, pady=10)

janela.mainloop()
```

- › strip() + if texto impede adicionar tarefas vazias
- › tk.END significa 'depois do último item' (para insert) e 'até o fim' (para delete)
- › Entry.get() lê, Entry.delete(0, END) limpa: par de operações que você usará sempre

Capítulo 6 — Projeto integrador

Exercício 16 — Sistema de reservas de hotel (POO + SQLite + pandas)

```
import sqlite3
import pandas as pd

class ReservaDB:
    def __init__(self):
        self.conn = sqlite3.connect(":memory:")
        self.conn.execute("""CREATE TABLE reservas (
            id INTEGER PRIMARY KEY,
            hospede TEXT,
            noites INTEGER,
            diaria REAL)""")

    def adicionar(self, hospede, noites, diaria):
        self.conn.execute(
            "INSERT INTO reservas (hospede, noites, diaria) VALUES (?, ?, ?)",
            (hospede, noites, diaria))
        self.conn.commit()

    def listar(self):
        return pd.read_sql("SELECT * FROM reservas", self.conn)

    def faturamento(self):
        total = self.conn.execute(
            "SELECT SUM(noites * diaria) FROM reservas").fetchone()[0]
        return total or 0.0

    def maior_reserva(self):
        linha = self.conn.execute(
            "SELECT hospede FROM reservas "
            "ORDER BY noites * diaria DESC LIMIT 1").fetchone()
        return linha[0] if linha else None
```

- › Cada classe tem uma responsabilidade: ReservaDB cuida da persistência e dos relatórios
- › A soma e o 'maior' são feitos pelo banco (SUM, ORDER BY ... LIMIT 1), não em loops Python
- › 'total or 0.0' trata o caso SUM() = NULL quando a tabela está vazia